

Embedded Systems

1. Microcontrollers

A microcontroller is a small computer on a single chip that can be programmed to control electronic devices. Common examples used in IoT:

- **ESP8266**: Has WiFi built-in, 18 GPIO pins, operates at 3.3V
- **ESP32**: More powerful than ESP8266, has WiFi + Bluetooth, more GPIO pins
- **ATmega328**: Found on Arduino boards, no built-in WiFi, operates at 5V

In the Arduino ecosystem, a program is called a **sketch**. Every sketch has two essential functions that form its backbone:

- **void setup()**: This is your initialization block. It runs only **once** when the microcontroller first boots up. You use it to set up things like Wi-Fi connections, configure pins, or initialize sensors.
- **void loop()**: This is the heart of your program. As soon as setup() is finished, loop() begins and runs **endlessly** in a cycle until the device is powered off or reset.

Microcontrollers interact with other electronic components through **GPIO** (General Purpose Input/Output) pins. These pins can be configured to either send signals out (output) or receive signals in (input).

Digital vs. Analog Pins

- **Digital Pins**: These pins operate on a two-state basis: **HIGH** (e.g., 3.3V or 5V) or **LOW** (0V). They are perfect for reading a button press or turning an LED on or off. You can set their mode (input or output) in your code.
- **Analog Pins**: These pins can read a range of voltage levels, not just HIGH or LOW. An analog-to-digital converter (ADC) inside the μC converts this voltage into a number (e.g., between 0 and 1023). This is ideal for sensors that measure variable phenomena like temperature or light intensity.

Key GPIO Functions

To use a pin, you first need to tell the μC its role in the `setup()` function:

- `pinMode(pin, MODE);` Sets a digital pin to be either `INPUT` or `OUTPUT`.

Then, in your `loop()`, you can read from or write to the pins:

- `digitalRead(pin)` ; Reads the state of a digital pin (returns HIGH or LOW).
- `digitalWrite(pin, STATE)` ; Sets a digital pin to HIGH or LOW.
- `analogRead(pin)` ; Reads the value from an analog input pin.

2. Device Communication Protocols

When you need to exchange more complex data than just a simple HIGH/LOW signal, you need a communication protocol. Both devices must agree on the protocol and its parameters (like speed) to understand each other. Here are the three main "historical" protocols used with microcontrollers.

Serial (UART)

Characteristics:

- Uses 2 pins: **RX** (receive) and **TX** (transmit)
- TX of one device connects to RX of other (and vice versa)
- No clock signal - both sides must agree on same speed (bauds)
- Point-to-point only (one device per connection)

Hardware vs Software Serial:

- **Hardware UART:** Built into μ C, faster, uses dedicated pins
- **SoftwareSerial:** Emulated by code, slower (~9600 bauds max), high CPU load

Example:

```
void setup() {

    Serial.begin(115200);  // For PC communication

    Serial2.begin(9600, SERIAL_8N1, 27, 28);  // RX=27, TX=28
}

void loop() {

    while (Serial2.available()) {

        Serial.print(char(Serial2.read()));

    }

}
```

Limitations:

- Needs separate pins for each device
- Speed typically limited to 115200 bauds (lower for software serial)

I2C (Inter-Integrated Circuit)

Characteristics:

- Shared bus - multiple devices on same 2 wires
- **SDA**: Data line
- **SCL**: Clock line (synchronizes all devices)
- Master-slave architecture
- Each slave has unique address (0-255)

Key Features:

- Default speed: 100 Kbits/s (can go up to >1 Mbits/s)
- Bidirectional communication
- Master can broadcast to all slaves

Arduino Usage:

```
#include <Wire.h>

Wire.begin(SDA_PIN, SCL_PIN);  // Initialize I2C

Wire.setClock(frequency);
```

Major Limitation: Device addresses are **fixed by manufacturer**. Example: BME280 sensor can only use 0x76 or 0x77, so max 2 on same bus!

SPI (Serial Peripheral Interface)

Characteristics:

- Uses 3 shared wires + 1 per device
- **MISO**: Master In, Slave Out
- **MOSI**: Master Out, Slave In
- **Clock**: Synchronization
- **CS/SS**: Chip Select (one per device, active LOW)

Advantages:

- Generally faster than I2C
- No address conflicts

Disadvantages:

- Consumes more GPIO pins (1 CS pin per device)
- More complex wiring (used through device specific libraries)

3. Interrupts and PWM

Interrupts

An **interrupt** is a signal that pauses the microcontroller's current task to immediately execute a special function, called an **interrupt service routine (ISR)** or callback function. This is extremely useful for reacting to events like a button press without having to constantly check the pin's state in the `loop()` function (a process called "polling").

- **How it works:** You "attach" an interrupt to a digital pin and specify a callback function to run when a certain change occurs (e.g., the pin signal going from `LOW` to `HIGH`, `HIGH` to `LOW`, or `CHANGE`).
- **Important Keywords:**
 - **volatile:** Any global variable modified inside an ISR must be declared with `volatile` to ensure it's always read from main memory, not a temporary cache.
 - **ICACHE_RAM_ATTR** (for ESPs): This macro forces the callback function itself to be stored in fast RAM, which is a requirement for ISRs.

Pulse Width Modulation (PWM)

PWM is a clever technique used to simulate an analog signal using a digital pin. It works by switching a digital pin ON and OFF very quickly.

- **Duty Cycle:** This is the key concept. It's the percentage of time the signal is HIGH (on) during one cycle.
 - A **25% duty cycle** means the pin is HIGH for 25% of the time and LOW for 75%. This would make an LED appear dim.
 - A **75% duty cycle** means the pin is HIGH for 75% of the time, making the LED appear brighter.
- **In Arduino:** The function is `analogWrite(pin, value)`. Despite the name "analog," it works on digital pins capable of PWM. The `value` determines the duty cycle.

4. WIFI Communication

Microcontrollers like the ESP8266 and ESP32 have built-in Wi-Fi capabilities.

Wi-Fi Modes

An ESP board can operate in several modes:

1. **Client / Station Mode (WIFI_STA):** This is the most common mode. The μ C connects to an existing Wi-Fi network (like your home router) and gets an IP address.
2. **Access Point Mode (WIFI_AP):** The μ C creates its own Wi-Fi network that other devices can connect to. It acts like a mini-router, assigning IP addresses, but doesn't have an IP address itself.

3. **Client + AP Mode (WIFI_AP_STA):** The μ C does both at the same time—it connects to an existing network while also providing its own access point.

WiFi Client Setup

```
#include <ESP8266WiFi.h> // For ESP8266
// #include <WiFi.h>      // For ESP32

void setupWifi() {
    const char *ssid = "your_network";
    const char *password = "your_password";

    WiFi.setAutoConnect(false); // Prevent auto-connection to
last network
    WiFi.setSleepMode(WIFI_NONE_SLEEP); // Keep WiFi always active
    WiFi.mode(WIFI_STA); // Client mode
    WiFi.begin(ssid, password);

    // Wait for connection
    while (WiFi.status() != WL_CONNECTED) {
        delay(500);
        Serial.print(".");
    }

    Serial.println(WiFi.localIP()); // Print assigned IP
}
```

WiFi Access Point Setup

```
void setupAP() {
    IPAddress local_IP(192, 168, 0, 1);
    IPAddress gateway(192, 168, 0, 1);
    IPAddress subnet(255, 255, 255, 0);

    WiFi.mode(WIFI_AP_STA); // AP + Client mode
    int channel = 1; // Channels 1-13 (keep 4 apart to avoid
interference)

    WiFi.softAP("MyNetwork", "password", channel);
    WiFi.softAPConfig(local_IP, gateway, subnet);
}
```

```

    Serial.println(WiFi.softAPIP());
}

```

Important: ESP as AP can handle max 4 clients by default. Not reliable for many connections!

TCP Client/Server Model

Key Concepts:

- **Reliable protocol:** Lost packets are resent, packets reordered if needed
- **Connection-oriented:** Handshake required before communication
- **Socket:** Endpoint for communication
- **Client-Server model:** Client initiates connection, server waits

Requirements:

- Both sides need: IP address + port number
- Server needs two sockets:
 - One for accepting connections
 - One for actual communication

Usage:

- **µC as a TCP Client:** The µC initiates a connection to a server at a specific IP address and port. It uses the `WiFiClient` class. After connecting, it can send (`client.println()`) and receive (`client.read()`) data.

```

#include <ESP8266WiFi.h>

WiFiClient client;
byte connState = 0;
const char* ipServ = "192.168.0.1";
int portServ = 12345;

// Blocking function to read a line
String readLine() {
    char buffer[1024];
    memset(buffer, 0, 1024);
    int index = 0;
    char c;

    while(true) {
        while (!client.available()) {} // Block until data available

```

```

        while ((index < 1023) && (client.available())) {
            c = client.read();
            if (c == '\n') return String(buffer);
            buffer[index++] = c;
        }

        if (index == 1023) return String(buffer);
    }
}

bool doConnection() {
    if (client.connect(ipServ, portServ)) {
        connState = 1;
        client.setNoDelay(true); // ESP8266 specific: disable
buffering
        return true;
    }
    return false;
}

void loop() {
    if (connState == 0) {
        if (!doConnection()) delay(1000);
    }

    if (connState == 1) {
        client.write(10);           // Send single byte
        client.println("Hello");    // Send text line
        String msg = readLine();    // Receive line (blocking)
    }
}

```

Key Points:

- `client.read()` reads ONE byte, non-blocking
- `client.available()` returns true if data ready
- `while(!client.available()) {}` creates blocking behavior (OK on μ C, forbidden on regular OS)

- **µC as a TCP Server:** The µC listens for incoming connections on a specific port. It uses the `WiFiServer` class. When a client connects, the server accepts the connection and can then communicate with that client. Due to limited resources, a µC can typically only handle one client at a time, not concurrently.

```
WiFiServer server(12345); // Listening socket on port 12345
WiFiClient client;
byte connState = 0;

void waitConnection() {
    client = server.available(); // Non-blocking check for incoming
    connection
    if (client) {
        connState = 1;
        client.setNoDelay(true); // ESP8266 specific
    }
}

void loop() {
    if (connState == 0) {
        waitConnection();
    }

    if (connState == 1) {
        while (!client.available()) {} // Wait for data

        // Read available data
        byte buf[1024];
        int idx = 0;
        while (client.available()) {
            buf[idx++] = client.read();
        }
    }
}
```

Important Limitation: µC servers cannot handle multiple clients concurrently (no multithreading like Java/C servers).

The Importance of a Communication Protocol

When a client (like your ESP32) talks to a server (like a Java application on a PC), they must agree on a set of rules for their conversation. This is their **communication protocol**.

- **Why it's needed:** A server might offer multiple services (e.g., get time, set time). The protocol defines how the client should structure its request so the server understands what is being asked. The identifier for the service should be sent first.
- **Example (Text-based):**
 - Client sends: "GETTIME time_zone"
 - Server understands this is a request for the time.
 - Server replies: "OK" followed by "hour minute second".

5. Power Management for Battery-Powered Devices

Wi-Fi is a major source of power consumption. ESP boards offer several sleep modes to save energy.

- **Modem-Sleep:** The default mode. The Wi-Fi radio periodically turns off for short intervals. This reduces power consumption to around 20mA when not actively transmitting, but can increase latency.
- **Light-Sleep:** The CPU is paused, but the RAM remains powered on, so the program state is not lost. The Wi-Fi is disconnected. Consumption drops to about 1mA.
- **Deep-Sleep:** This is the most power-efficient mode. The CPU and most of the RAM are shut down. Power consumption can drop to just a few microamps ($\sim 10\mu\text{A}$).

Deep-Sleep Explained

When a device enters deep-sleep, it's almost completely off.

- **Waking Up:** It can be configured to wake up after a set amount of time or via an external signal on a GPIO pin.
- **The Catch:** Upon waking, the μC **reboots**. This means all data stored in the main RAM is **lost**.
- **Preserving Data:** To save small amounts of data (like a boot counter) across reboots, you can use the special **RTC RAM**, which stays powered on during deep-sleep.

Deep-Sleep Implementation in ESP8266

```
uint32_t bootcount;
```

```
void setup() {
```

```
    // Read from RTC RAM (survives deep sleep)
```

```

    ESP.rtcUserMemoryRead(0, &bootcount, sizeof(bootcount));

    bootcount++;

    // Write back to RTC RAM

    ESP.rtcUserMemoryWrite(0, &bootcount, sizeof(bootcount));
}

void loop() {

    ESP.deepSleep(5000000); // Sleep 5 seconds (in microseconds)

}

```

Deep-Sleep Implementation in ESP32

```

RTC_DATA_ATTR int bootCount = 0; // Stored in RTC RAM

void setup() {

    bootCount++;

    // Configure wake-up sources

    esp_sleep_enable_ext0_wakeup(GPIO_NUM_33, 1); // External pin
    (1=HIGH)

    esp_sleep_enable_timer_wakeup(5000000); // Timer
    (microseconds)

}

void loop() {

    esp_deep_sleep_start(); // Enter deep sleep

}

```

ESP32 Advantages:

- No pin connection needed for timer wake-up
- Multiple wake-up pins (0, 2, 4, 12-15, 25-27, 32-39)
- Can combine timer + external wake-up
- Can detect which source caused wake-up

6. Java TCP Socket Programming

Key Java Classes

- **Socket:** For communication (both client and server use this)
- **ServerSocket:** For waiting/accepting connections (server only)

Important: Java Socket class has NO direct send/recv methods! Must use streams.

Stream Types in Java

Byte Streams:

- `getInputStream()`: Returns `InputStream` for reading bytes
- `getOutputStream()`: Returns `OutputStream` for writing bytes
- Good for files, but tedious for values/text

Character/Object Streams:

- Can wrap byte streams for higher-level operations
- **BufferedReader:** Read lines of text
- **PrintStream:** Write lines of text
- **ObjectStreams:** Java-specific, NOT universal (don't use for IoT!)

Java Server Example (Echo Server)

```
import java.io.*;
import java.net.*;

class MyServer {
    public static void main(String[] args) {
        ServerSocket sockConn = null; // Connection socket
        Socket sockComm = null;       // Communication socket

        try {
            int port = Integer.parseInt(args[0]);
            sockConn = new ServerSocket(port);

            sockComm = sockConn.accept(); // Wait for client
            (blocking)

            // Wrap streams for text communication
            PrintStream ps = new
            PrintStream(sockComm.getOutputStream());
```

```

        BufferedReader br = new BufferedReader(
            new InputStreamReader(sockComm.getInputStream())
        );

        String msg = br.readLine(); // Receive (blocking)
        ps.println(msg);           // Echo back

    } catch(IOException e) {
        System.err.println("Socket error: " + e.getMessage());
    }
}
}

```

Java Client Example

```

import java.io.*;
import java.net.*;

class MyClient {
    public static void main(String[] args) {
        Socket sockComm = null;

        try {
            String ipServ = args[0];
            int port = Integer.parseInt(args[1]);

            sockComm = new Socket(ipServ, port); // Connect to server

            PrintStream ps = new
PrintStream(sockComm.getOutputStream());
            BufferedReader br = new BufferedReader(
                new InputStreamReader(sockComm.getInputStream())
            );

            ps.println(args[2]); // Send message
            String msg = br.readLine(); // Receive response
(blocking)
            System.out.println("Server answers: " + msg);

```

```

        } catch(IOException e) {
            System.err.println("Socket error: " + e.getMessage());
        }
    }
}

```

Key Differences from μ C:

- Java has proper blocking `readLine()` method
- Must wrap streams for text communication
- Exception handling required

Exercise 1: Basic Button Control For LED

This exercise demonstrates the most fundamental way to read a digital input (a button) and control a digital output (an LED). It introduces the concept of **polling**, where the program constantly checks the button's state inside the **loop()**.

Full Code Solution

```

#include <ESP8266WiFi.h>

#define PIN_LED D1
#define PIN_BUTTON D2

byte state;
byte oldState;

void setup() {
    Serial.begin(115200);
    while(!Serial);

    // Power off Wifi to save energy
    WiFi.mode(WIFI_OFF);
    WiFi.forceSleepBegin();

    pinMode(PIN_BUTTON, INPUT); // Set button pin as input
    pinMode(PIN_LED, OUTPUT);   // Set LED pin as output

    digitalWrite(PIN_LED, LOW); // Switch off the LED at start
    state = digitalRead(PIN_BUTTON);
    oldState = state;
}

void testSwitch() {
    state = digitalRead(PIN_BUTTON);
    if (state != oldState) { // Only act if the state has changed
        if (state == 0) { // LOW means button is released in a pull-down
            setup
            Serial.println("switch off => light off");
        }
    }
}

```

```

        digitalWrite(PIN_LED, LOW);
    } else { // HIGH means button is pressed
        Serial.println("switch on => light on");
        digitalWrite(PIN_LED, HIGH);
    }
    oldState = state; // Update the old state for the next loop
}
}

void loop() {
    testSwitch();
    delay(10); // A small delay to prevent the µC from crashing
}

```

How It Works

- **Polling:** The `loop()` function continuously calls `testSwitch()`. This method of repeatedly checking an input is called polling.
- **State Change Detection:** To avoid printing messages and changing the LED state on every single loop, the code uses two variables: `state` and `oldState`. The `if (state != oldState)` condition ensures that the code inside only runs when the button is actually pressed or released.
- **Energy Saving:** In `setup()`, `WiFi.mode(WIFI_OFF)` is called to disable the Wi-Fi module, which is a major source of power consumption and isn't needed for this exercise.

Exercise 2: Using Interrupts For Button Control

This solution improves on the first by using an **interrupt** instead of polling. This is a much more efficient way to handle external events.

Full Code Solution

```

#include <ESP8266WiFi.h>

#define PIN_LED D1
#define PIN_BUTTON D2

volatile byte state; // 'volatile' is crucial for variables used in an
ISR

// The Interrupt Service Routine (ISR)
void ICACHE_RAM_ATTR testSwitch() {
    state = digitalRead(PIN_BUTTON);
    if (state == 0) {
        digitalWrite(PIN_LED, LOW);
    } else {
        digitalWrite(PIN_LED, HIGH);
    }
}

void setup() {
    Serial.begin(115200);
    while(!Serial);
}

```

```

WiFi.mode(WIFI_OFF);
WiFi.forceSleepBegin();
delay(10);

pinMode(PIN_BUTTON, INPUT);
pinMode(PIN_LED, OUTPUT);

digitalWrite(PIN_LED, LOW);
state = digitalRead(PIN_BUTTON);

// Attach the interrupt
attachInterrupt(digitalPinToInterrupt(PIN_BUTTON), testSwitch,
CHANGE);
}

void loop() {
  delay(1000); // The loop is now mostly empty, just waiting
}

```

How It Works

- **Event-Driven Logic:** Instead of the `loop()` constantly checking the button, the `attachInterrupt()` function in `setup()` tells the microcontroller to automatically run the `testSwitch()` function whenever the voltage on `PIN_BUTTON` changes (`CHANGE`).
- **Efficiency:** The `loop()` function can now do other things or simply wait (`delay(1000)`), as it no longer needs to worry about the button. This makes the program more responsive and efficient.
- **ISR Requirements:**
 - The function `testSwitch()` is now an Interrupt Service Routine (ISR). On ESP boards, it must be preceded by `ICACHE_RAM_ATTR` to ensure it's loaded into fast RAM.
 - The global variable `state`, which is modified inside the ISR, must be declared as `volatile` to prevent the compiler from making optimizations that could lead to incorrect behavior.

Exercise 3: State Machines & Blinking Game

This exercise introduces a more advanced programming structure: **a state machine**. This is a powerful way to manage programs that have distinct phases of operation.

Full Code Solution

```

#include <ESP8266WiFi.h>

#define PIN_LED D1
#define PIN_BUTTON D2
#define STATE_BLINK 0
#define STATE_BUTTON 1

```

```

int state;
volatile int countButton;
int nbBlinks;
long t1; // Time when blink sequence ends
volatile long t2; // Time when button is pressed enough times
volatile boolean butStop;

void ICACHE_RAM_ATTR testSwitch() {
    countButton++;
    if (countButton >= (2 * nbBlinks)) { // A press and release counts as
2 changes
        t2 = millis();
        butStop = true; // Signal to the main loop that this phase is over
    }
}

void doBlinks(int min, int max, int duration, boolean sameLast = false)
{
    nbBlinks = random(min, max);
    // ... [Logic for blinking the LED 'nbBlinks' times] ...
}

void setup() {
    Serial.begin(115200);
    while(!Serial);

    WiFi.mode(WIFI_OFF);
    WiFi.forceSleepBegin();
    pinMode(PIN_BUTTON, INPUT);
    pinMode(PIN_LED, OUTPUT);
    digitalWrite(PIN_LED, LOW);

    state = STATE_BLINK; // Start in the BLINK state
    attachInterrupt(digitalPinToInterrupt(PIN_BUTTON), testSwitch,
CHANGE);
}

void loop() {
    if (state == STATE_BLINK) {
        Serial.println("be ready...");
        delay(5000);
        Serial.println("count the blinks");
        doBlinks(3, 6, 500);
        Serial.println("now push the button");
        t1 = millis();

        // Transition to the next state
        state = STATE_BUTTON;
        butStop = false;
        countButton = 0;
    } else { // state == STATE_BUTTON
        if (butStop) { // Check the flag set by the ISR
            digitalWrite(PIN_LED, HIGH);
            delay(200);
            digitalWrite(PIN_LED, LOW);
            Serial.println("time to react: " + String(t2 - t1) + "ms");

```



```

        // Transition back to the initial state
        state = STATE_BLINK;
    } else {
        delay(5);
    }
}
}
}

```

How It Works

- **State Variable:** A global variable `state` is used to track the program's current mode (`STATE_BLINK` or `STATE_BUTTON`). The `loop()` function uses an `if/else` structure to execute different code based on the value of `state`.
- **State Transitions:** After a task is completed in one state (e.g., the blinking sequence finishes), the code changes the value of the `state` variable to move the program into the next phase.
- **Communication between `loop` and ISR:** The interrupt handler (`testSwitch`) can't perform long tasks. Instead, when the required number of button presses is reached, it sets a boolean "flag" (`butStop = true`). The main `loop()` continuously watches this flag. When it sees the flag is true, it knows the user has completed the task and it can proceed.

Exercise 4: Piezo Sensor + TCP Client

This exercise demonstrates how to connect an ESP8266 to a Wi-Fi network and communicate with a custom TCP server to send sensor data.

Full Code Solution

```

#include <ESP8266WiFi.h>

// WiFi & Server related
WiFiClient client;
byte connState;
const char *ssid = "testingiot";
const char *password = "testingiot";
char ipServ[20] = "199.168.0.2";
int portServ = 12345;
String idClient;

#define PIN_PIEZO A0
int piezoLevel;

void setupWifi() {
    WiFi.mode(WIFI_STA); // Set as a Wi-Fi client
    WiFi.begin(ssid, password);
    while (WiFi.status() != WL_CONNECTED) {
        delay(500);
        Serial.print(".");
    }
    Serial.print("Connected, IP address: ");
}

```

```

    Serial.println(WiFi.localIP());
}

bool setupServerConnection() {
    if (client.connect(ipServ, portServ)) {
        connState = 1;
        Serial.println("connected to server");
        client.setNoDelay(true); // Disable buffering for small messages
        idClient = readLineServer(); // Server sends a unique ID upon
connection
        Serial.println("my id is " + idClient);
        return true;
    } else {
        connState = 0;
        Serial.println("connection failed");
        return false;
    }
}

String readLineServer() {
    // ... [Code to read a full line of text from the server] ...
}

void sendPiezoLevel() {
    String req = "PIEZO " + idClient + " " + piezoLevel;
    Serial.println("sending request: " + req);
    client.println(req);
}

void setup() {
    Serial.begin(115200);
    setupWifi();
    delay(100);
}

void loop() {
    if (connState == 0) {
        if (!setupServerConnection()) {
            delay(1000);
        }
    } else {
        piezoLevel = analogRead(PIN_PIEZO);
        if (piezoLevel >= 500) {
            sendPiezoLevel();
            String answer = readLineServer();
            if (answer.equals("OK")) {
                answer = readLineServer(); // Read the second line containing
the result
                Serial.println("servers answers: " + answer);
            } else {
                Serial.println("server reports an error: " + answer);
            }
            delay(500);
        }
    }
    delay(3); // Mandatory delay between analogRead() calls on ESP8266
}

```

How It Works

- **Connection Process:** In `setup()`, the `setupWifi()` function connects the ESP8266 to the specified Access Point. The `loop()` then attempts to connect to the TCP server using `setupServerConnection()`.
- **Client ID:** Immediately after connecting, the server sends the client a unique ID. The client must read this ID and include it in all subsequent requests so the server knows who is sending the data.
- **Request Protocol:** The client sends data to the server using a predefined text format: `"PIEZO client_id piezo_value"`. This structured message is the **protocol** that both client and server have agreed to use.
- **Server Response:** The server replies with `"OK"` if the request is valid, followed by a second line with the calculated data. If the request is invalid, it sends an `"ERR"` message. The client code must be able to handle both of these cases.
- **Hardware Constraint:** The `delay(3)` at the end of the loop is critical. The ESP8266 requires a small delay between consecutive `analogRead()` calls to function correctly.